



|                  |                                  |
|------------------|----------------------------------|
| <b>Status</b>    | Finished                         |
| <b>Started</b>   | Saturday, 13 June 2026, 11:46 AM |
| <b>Completed</b> | Saturday, 13 June 2026, 11:59 AM |
| <b>Duration</b>  | 12 mins 21 secs                  |
| <b>Grade</b>     | Not yet graded                   |

**Question 1**

Complete

Marked out of 24.00

In Experiment B, the AP system returned a stale value immediately after a write. How many seconds did it take to converge to the correct value? Which line of code in `sync.go` controls this? What would happen if you changed it to 10 seconds?

In the current setup, the AP store is configured to sync every 1 second, so the stale value should converge on the next sync tick, usually about 1 second after the write.

`syncInterval: 1 * time.Second,`

If we changed that interval to 10 seconds, the AP nodes would only exchange state every 10 seconds, so stale reads could persist much longer. In experiment, the second read after 3 seconds would likely still show the old value unless a sync happened to occur earlier by timing luck

 [\\_node.go](https://node.go)

**Question 2**

Complete

Marked out of 24.00

In Experiment D, the CP system refused writes when 3 nodes were down. The quorum formula is  $\text{len(peers)}/2 + 1 = 3$ . Why is this threshold 3 and not 2? What problem would occur if you changed the quorum to 2?

The threshold is 3 because the system has 5 nodes total, and a quorum must be a strict majority. That guarantees any two quorums overlap by at least one node, which is what keeps the CP store consistent after failures or partitions.

If we changed the quorum to 2, writes would become too permissive. Two different partitions could each accept different values on different pairs of nodes, and later reads could see conflicting or stale data after recovery.

**Question 3**

Complete

Marked out of 24.00

You used `time.Now().UnixNano()` as a version number for conflict resolution in the AP system. Describe a realistic scenario where this approach gives the wrong result. How would vector clocks solve this problem?

A realistic failure case is clock skew across nodes. For example, node A's clock is 5 seconds ahead of node B's. If B actually receives and stores a write later in real time, but A's timestamp is larger because its clock runs fast, your last-write-wins merge in `store.go` will incorrectly keep A's version even though B's write happened later. The same problem can happen if NTP adjusts one machine's clock backward or if two nodes write concurrently and physical time does not reflect causality.

Vector clocks avoid that by tracking version history per node instead of a single wall-clock number. Each write carries a vector like {A: 3, B: 1, C: 0}. When merging, nodes can tell whether one update definitely happened after another, or whether the writes are concurrent. In the concurrent case, vector clocks do not pretend one is "newer" just because of wall time; they preserve the conflict so the application can resolve it correctly. That is the key difference from `time.Now().UnixNano()` in `node.go`.

**Question 4**

Complete

Marked out of 24.00

A hospital patient record system vs a Twitter like counter — which should be CP and which AP? Use specific results from your Experiments B and D to justify your answer.

Hospital patient record system - CP

A hospital system should be CP because data must always be accurate. In Experiment B, CP immediately returned the latest value, while AP returned a stale result (not found). In Experiment D, CP refused writes when quorum was not available, protecting consistency.

Twitter like counter - AP

A Twitter like counter should be AP because availability is more important than perfect consistency. In Experiment B, AP briefly returned a stale result but became correct after a few seconds. In Experiment D, AP continued accepting writes even when three nodes were down.

Conclusion:

Hospital records : CP

Twitter likes : AP.

**Question 5**

Complete

Not graded

Record your results from each experiment below.

| Experiment                       | AP Result                                            | CP Result                        |
|----------------------------------|------------------------------------------------------|----------------------------------|
| A – All gets succeeded?          | <input type="text" value="yes"/>                     | <input type="text" value="yes"/> |
| B – Immediate read from node5    | <input type="text" value="no"/>                      | <input type="text" value="yes"/> |
| B – Read after 3 seconds         | <input type="text" value="yes"/>                     | <input type="text" value="yes"/> |
| C – Write with 2 nodes down?     | <input type="text" value="yes"/>                     | <input type="text" value="yes"/> |
| D – Write with 3 nodes down?     | <input type="text" value="yes"/>                     | <input type="text" value="no"/>  |
| D – Error message (if any)       | <input type="text" value="✗ key='alert' not found"/> | N/A                              |
| D – Recovery time (seconds)      | <input type="text" value="1527 ms"/>                 | <input type="text" value="N/A"/> |
| E – Winning value after conflict | <input type="text" value="999"/>                     | N/A (CP cannot conflict)         |

**Question 6**

Complete

Not graded

Fill in the comparison table based on your experimental observations.

| Property                          | AP System                                                            | CP System                                                                      |
|-----------------------------------|----------------------------------------------------------------------|--------------------------------------------------------------------------------|
| Always accepts writes             | <input type="text" value="yes"/>                                     | <input type="text" value="no"/>                                                |
| Read always returns latest value  | <input type="text" value="no"/>                                      | <input type="text" value="yes"/>                                               |
| Handles conflicting writes        | <input type="text" value="yes"/>                                     | <input type="text" value="no"/>                                                |
| Works when majority of nodes down | <input type="text" value="yes"/>                                     | <input type="text" value="no"/>                                                |
| Best use case                     | <input type="text" value="Social media likes, DNS, shopping carts"/> | <input type="text" value="Bank accounts, inventory systems, medical records"/> |

